

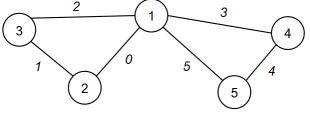
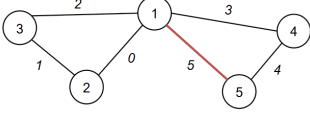
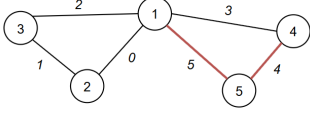
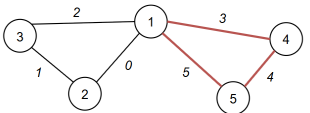
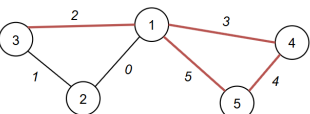
Report Graph Theory

Sample test used for explanation :

5 6 #Number of nodes (5) and edges (6)
 1 2
 2 3
 3 1
 1 4
 4 5
 5 1

Algorithms

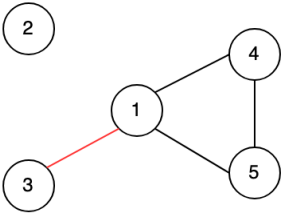
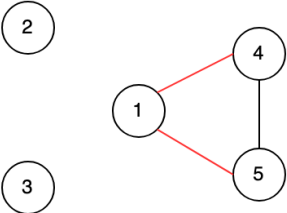
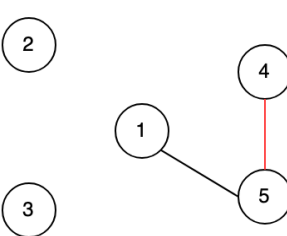
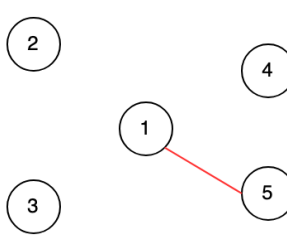
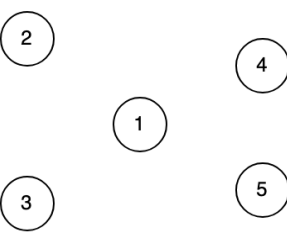
Hierholzer's algorithm

Step	node (u)	Used Edge	curr_path	circuit	Graph
0	-	[F, F, F, F, F, F]	[1]	[]	
1	5	[F, F, F, F, F, T]	[1, 5]	[]	
2	4	[F, F, F, F, T, T]	[1, 5, 4]	[]	
3	1	[F, F, F, T, T, T]	[1, 5, 4, 1]	[]	
4	3	[F, F, T, T, T, T]	[1, 5, 4, 1, 3]	[]	

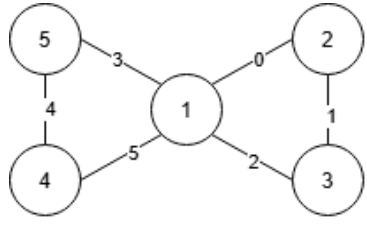
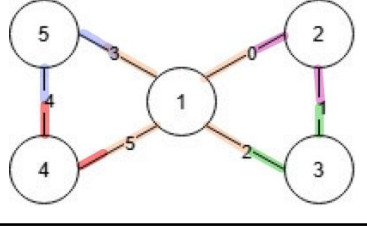
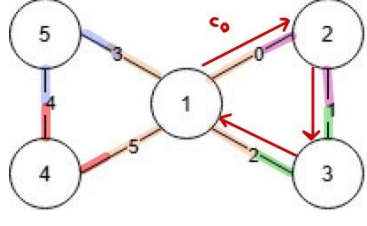
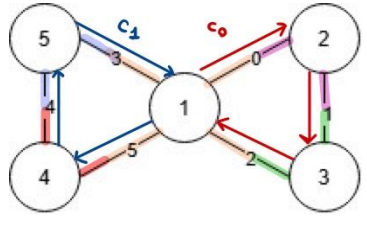
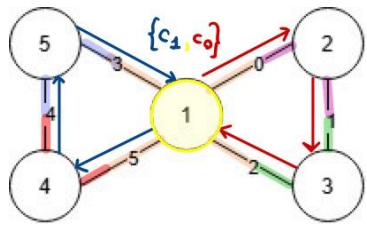
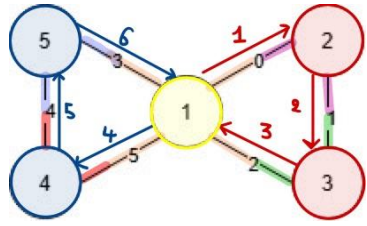
5	2	[F, T, T, T, T, T]	[1, 5, 4, 1, 3, 2]	[]	
6	1	[T, T, T, T, T, T]	[1, 5, 4, 1, 3, 2, 1]	[]	
7	2	[T, T, T, T, T, T]	[1, 5, 4, 1, 3, 2]	[1]	
8	3	[T, T, T, T, T, T]	[1, 5, 4, 1, 3]	[1, 2]	
.	.	.	.	.	.
13	1	[T, T, T, T, T, T]	[]	[1, 2, 3, 1, 4, 5, 1]	

Fleury's algorithm

Steps	Current node	Available Edges	Remaining edges	Bridge test and selection	Graph	Eulerian path
0	1	(1, 2) : v (3, 1) (1, 4) (5, 1)	(1, 2) (2, 3) (3, 1) (1, 4) (4, 5) (5, 1)	(1, 2) : not a bridge (1, 2) : Chosen		[1]
1	2	(2, 3) : v	(2, 3) (3, 1) (1, 4) (4, 5) (5, 1)	(2, 3) : Bridge but only alternative (2, 3) : Chosen		[1, 2]

2	3	(3, 1) : V	(3, 1) (1, 4) (4, 5) (5, 1)	(3, 1) : Bridge but only alternative (3, 1) : Chosen		[1, 2, 3]
3	1	(1, 4) : V (5, 1)	(1, 4) (4, 5) (5, 1)	(1, 4) : Not a bridge (1, 4) : Chosen		[1, 2, 3, 1]
4	4	(4, 5) : V	(4, 5) (5, 1)	(4, 5) : Bridge but only alternative (4, 5) : Chosen		[1, 2, 3, 1, 4]
5	5	(5, 1) : V	(5, 1)	(5, 1) : Bridge but only alternative (5, 1) : Chosen		[1, 2, 3, 1, 4, 5]
6	1	-	-	-		[1, 2, 3, 1, 4, 5, 1]

Tucker's Algorithm

Step	Phase	Vertex	Pairing / Action	Circuits / DSU State	Result	Graph
0	Validation	-	Check even degrees and connectivity from vertex 1	No circuit yet	The graph is Eulerian	
1	Local pairing	1-5	v1 : (e0,e2), (e3,e5); v2 : (e0,e1); v3 : (e1,e2); v4 : (e3,e4); v5 : (e4,e5)	No circuit identified yet	every incident edge has a partner	
2	Circuit detection	1	Follows e0 -> e1 -> e2	{c0}	c0 = 1 -> 2 -> 3 -> 1	
3	{C-F} = 9	1	Follows e3 -> e4 -> e5	{c0}, {c1}	c1 = 1 -> 4 -> 5 -> 1	
4	{A-B} = 7	1	Cross (e0,e2) with (e3,e5) to obtain (e0,e3) and (e2,e5)	Before: {c0}, {c1}	After DSU union : {c0,c1}	
5	{B-D} = 7	1	Follow the final partner relationships	One merged component	1 -> 2 -> 3 -> 1 -> 5 -> 4 -> 1	

Complexity Comparison

Algorithm	Main strategy	Time complexity	Space complexity
Fleury	Avoid bridges while constructing the route	$O(E^2)$	$O(V + E)$
Hierholzer	Construct and splice circuits	$O(V + E)$	$O(V + E)$
Tucker	Pair edges and merge circuits using DSU	$O(V + E * \alpha(E))$	$O(V + E)$

Hierholzer's algorithm is the best choice for this problem in most practical situations. It has optimal linear time complexity and is simpler to implement than Tucker's algorithm. Tucker's algorithm is also practically linear, but it requires additional structures for half-edges, pairings, and circuit merging. Fleury's algorithm is intuitive, but its repeated bridge tests make it the least efficient of the three.